

Name: Ashrith Peechara	Roll No: 2311CS040137
Course: Applied Agentic AI (LLM & RAG)	Assignment: Coding Assignment 1

Coding Assignment 1 - Applied Agentic AI (LLM & RAG)

1. LLM Workflow: Develop a Python program that accepts user input and generates a response using an LLM (OpenAI/Gemini/Ollama/Groq).

```
import os
from dotenv import load_dotenv
from groq import Groq

load_dotenv()
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

def get_response(user_input):
    chat_completion = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[
            {"role": "system", "content": "You are a helpful assistant."},
            {"role": "user", "content": user_input}
        ]
    )
    return chat_completion.choices[0].message.content

if __name__ == "__main__":
    user_input = input("Enter your question: ")
    response = get_response(user_input)
    print("\nLLM Response:\n", response)
```

2. Prompt Chaining: Implement a multi-step LLM workflow to generate a summary, extract key points, and produce three questions from a given topic.

```
import os
from dotenv import load_dotenv
from groq import Groq

load_dotenv()
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

def ask(prompt):
    response = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[{"role": "user", "content": prompt}]
    )
    return response.choices[0].message.content

def summarize(topic):
    prompt = f"Write a short summary about: {topic}"
    return ask(prompt)

def extract_key_points(summary):
    prompt = f"Extract the key points from this summary:\n{summary}"
    return ask(prompt)

def generate_questions(key_points):
    prompt = f"Based on these key points, generate 3 questions:\n{key_points}"
    return ask(prompt)

if __name__ == "__main__":
```

```
topic = input("Enter a topic: ")
summary = summarize(topic)
print("\nStep 1 - Summary:\n", summary)
key_points = extract_key_points(summary)
print("\nStep 2 - Key Points:\n", key_points)
questions = generate_questions(key_points)
print("\nStep 3 - Three Questions:\n", questions)
```

3. Agentic AI: Build a simple AI agent that accepts a task, plans the required steps, executes them, and displays the final output.

```
import os
from dotenv import load_dotenv
from groq import Groq

load_dotenv()
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

def ask(prompt):
    response = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[{"role": "user", "content": prompt}]
    )
    return response.choices[0].message.content

def plan_task(task):
    prompt = f"Break the following task into a numbered list of clear steps:\n{task}"
    return ask(prompt)

def execute_steps(task, plan):
    prompt = (
        f"Task: {task}\n"
        f"Plan:\n{plan}\n"
        "Now execute each step above and produce the final combined output."
    )
    return ask(prompt)

class SimpleAgent:
    def __init__(self, task):
        self.task = task
        self.plan = None
        self.result = None

    def run(self):
        print("Planning task...")
        self.plan = plan_task(self.task)
        print("\nPlan:\n", self.plan)
        print("\nExecuting steps...")
        self.result = execute_steps(self.task, self.plan)
        print("\nFinal Output:\n", self.result)
        return self.result

if __name__ == "__main__":
    task = input("Enter a task for the agent: ")
    agent = SimpleAgent(task)
    agent.run()
```

4. RAG-Based Question Answering: Develop a basic RAG application that retrieves relevant information from a PDF/TXT document and answers user queries using an LLM.

```
import os
from dotenv import load_dotenv
```

```
from groq import Groq
from pypdf import PdfReader

load_dotenv()
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

def load_document(file_path):
    if file_path.lower().endswith(".pdf"):
        reader = PdfReader(file_path)
        text = ""
        for page in reader.pages:
            text += page.extract_text() or ""
        return text
    else:
        with open(file_path, "r", encoding="utf-8") as f:
            return f.read()

def chunk_text(text, chunk_size=500):
    words = text.split()
    chunks = []
    for i in range(0, len(words), chunk_size):
        chunks.append(" ".join(words[i:i + chunk_size]))
    return chunks

def retrieve_relevant_chunks(query, chunks, top_k=3):
    query_words = set(query.lower().split())
    scored_chunks = []
    for chunk in chunks:
        chunk_words = set(chunk.lower().split())
        score = len(query_words & chunk_words)
        scored_chunks.append((score, chunk))
    scored_chunks.sort(key=lambda x: x[0], reverse=True)
    return [chunk for score, chunk in scored_chunks[:top_k]]

def answer_query(query, context_chunks):
    context = "\n\n".join(context_chunks)
    prompt = (
        f"Answer the question using only the context below.\n\n"
        f"Context:\n{context}\n\n"
        f"Question: {query}\n"
        "Answer:"
    )
    response = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[{"role": "user", "content": prompt}]
    )
    return response.choices[0].message.content

if __name__ == "__main__":
    file_path = input("Enter path to PDF/TXT document: ")
    document_text = load_document(file_path)
    chunks = chunk_text(document_text)
    query = input("Enter your question: ")
    relevant_chunks = retrieve_relevant_chunks(query, chunks)
    answer = answer_query(query, relevant_chunks)
    print("\nAnswer:\n", answer)
```
